



VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches

Hao Sun ^{a, b} , Lei Cui ^a , Lun Li ^a , Zhenquan Ding ^a , Zhiyu Hao ^a , Jiancong Cui ^{a, c} ,
Peng Liu ^d 

Show more 

 Share  Cite

<https://doi.org/10.1016/j.cose.2021.102417> 

[Get rights and content](#) 

Abstract

Vulnerability detection using machine learning is a hot topic in improving software security. However, existing works formulate detection as a classification problem, which requires a large set of labelled data while capturing semantical and syntactic similarity. In this work, we argue that similarity in the view of vulnerability is the key in detecting vulnerabilities. We prepare a relatively smaller data set composed of both vulnerabilities and associated patches, and attempt to realize security similarity from (i) the similarity between pair of vulnerabilities and (ii) the difference between a pair of vulnerability and patch. To achieve this, we setup the detection model using the Siamese network cooperated with BiLSTM and Attention to deal with source code, Attention network to improve the detection accuracy. On a data set of 876 vulnerabilities and patches of OpenSSL and Linux, the proposed model (VDSimilar) achieves about 97.17% in AUC value of OpenSSL (where the Attention network contributes 1.21% than BiLSTM in Siamese), which is more outstanding than the most advanced methods based on deep learning.

Introduction

Vulnerability denotes a weakness, defect or security bug in a software program. It is introduced due to design error, low coding quality, or insufficient security testing. The vulnerability can be directly used by a hacker to gain access to a system or network. Thus, vulnerability detection is essential in improving software security.

Code similarity is one promising method to detect vulnerabilities when some known vulnerabilities are given. This is because it is recognized that some vulnerable code share the same class of

weakness, e.g., memory buffer overflow, improper input validation, out-of-bounds write. The basic idea of code similarity is to compare the currently known vulnerabilities and code of test programs for finding matches. For example, many approaches first transform the code into an unit¹ of intermediate representation (e.g., tokens, trees, or graphs), and then employ a comparison algorithm to find matched units to known vulnerable unit (Hunt, MacIlroy, 1976, Jiang, Misherghi, Su, Glondou, 2007, Kamiya, Kusumoto, Inoue, 2002, Kim, Woo, Lee, Oh, 2017, Su, Tan, Xiong, Ji, Shi, Liu, 2017, Vinyals, Toshev, Bengio, Erhan, 2015). However, due to the large variance of source code in both syntax and semantics, they suffer low accuracy and coverage in detect real-world vulnerabilities. Recently, with the rapid development of deep learning, data-driven vulnerability detection approaches have received much Attention. They view vulnerability detection as a classification problem and train a classifier on vulnerabilities databases such as CVE Details, NVD, and SARD. In addition, some recent works attempt to employ machine learning methods to compute the similarity between two units transformed from the source code, e.g., graphs (Lietal., 2019). Contributed to the advance of deep learning, these methods have shown the effectiveness in vulnerability detection.

Unfortunately, current code similarity based methods intend to detect vulnerability that is semantically and syntactically similar to a known vulnerability. However, semantical and syntactic similarity does not guarantee to find vulnerabilities. This is because the vulnerable snippet often takes a little fragment of the entire vulnerable function, i.e., the snippet may only contain a few lines or even one line of code, while the function involves dozens to hundreds of lines. Therefore, for two functions that are likely to identical in syntax and semantics, even subtle differences can lead to different categories in the view of vulnerability (i.e., one is vulnerable while another is benign), as we will detail in Fig.2. Thus, the main property that we want a good detection solution to satisfy is to capture the similarity in the view of vulnerability, rather than simply syntax and semantics.

On the other hand, deep learning-based methods always require a large data set to train a model. For example, VulDeePecker Lietal.(2018b) is performed on 61,638 code gadgets for vulnerability detection. Although it is possible to prepare such a large data set, it requires heavy human efforts. Thus, we want a method that can perform deep learning-based detection on a relatively small data set.

In this paper, we satisfy the property by presenting a metric learning-based approach, which trains a code similarity detector on a data set of vulnerabilities and patches. In particular, we pay our efforts in two directions, a data set that characterizes vulnerabilities and a metric learning model that learns similarity in the view of vulnerability. First, we prepare a data set of CVE bunches,² each of which contains multiple vulnerable functions and patched functions associated with one CVE, as illustrated in Fig.1. These functions for each CVE can be acquired from various versions of the affected software program, and they follow two rules.

a) For two vulnerable functions of the same CVE across two program versions, the vulnerability snippet will keep despite of the code change. b) For one vulnerable function and the associated patched function, the vulnerability snippet will disappear no matter how the code changes. Therefore, each bunch of functions potentially provide vulnerability characteristics of one CVE. Second, in the view of vulnerability, two vulnerable functions across different versions should be

treated as similar, even they experience code changes. On the other hand, a vulnerable function and its patched function, even looks similar in syntax, should be treated as different since the vulnerability snippet has been removed. Inspired by this, we employ the Siamese model to learn the similarity between two vulnerable functions, while learning the difference between vulnerable function and patched function. In addition, we incorporate BiLSTM and Attention network in the Siamese model, which helps generate more accurate and focused representations of functions for detection. In this way, the trained model can perceive similarity in the view of vulnerability instead of merely semantics and syntax.

Fig.1 compares our approach with existing works. First, existing works requires a large set of functions (e.g., about **1M** functions in Russelletal., 2018), each of which is labelled as vulnerable or not. In comparison, our prepared data set contains a relatively smaller set of both vulnerable functions and patched functions without labelling. By pairing the functions, we naturally augment the training data. Second, most of existing methods view the vulnerability detection as a classification problem, while we pay attention to the code similarity with a metric learning model incorporated with Attention network. The main advantage of our model is that it can pay attention to the snippets similarity in terms of vulnerability similarity rather than syntactic and semantical similarity of the entire functions.

To summarize, we make the following contributions.

- First, a data set containing real-world vulnerable functions and associated patched functions of 147 OpenSSL and Linux CVEs, which helps characterize vulnerability snippets. The data set is available in Github.³
- Second, a detection model using Siamese network cooperated with BiLSTM and Attention network. By taking pairs of vulnerable functions and patched functions, the model is able compute the similarity between two functions, so as to detect vulnerabilities. We plan to release the source code upon the publication of this paper.
- Third, the system implementation and evaluation on the data set to prove the effectiveness of the proposed model.

In the next section, we provide a brief introduction to several important notations and summarize related work from two aspects: traditional methods and machine learning methods. In Section3, we present the basic idea that uses the vulnerability and patches for the similarity comparison. In Section4, we describe the technical details of VDSimilar, including data preparation, the detection model of VDSimilar and the process of detecting vulnerabilities. In Section5, we compare VDSimilar against several existing works to show its effectiveness. In Section6, we conclude our work and discuss the future work.

Access through your organization

Check access to the full text by signing in through your organization.

Section snippets

Background and related work

We first present several important notations used in the paper. Then we give a brief introduction to code similarity-based detection approach and review existing works in this literature. ...

Why similarity of vulnerability snippet

We first explain why the snippet rather than the entire function is the key to detect vulnerabilities.

Fig.2 shows a code fragment of *tls_decrypt_ticket* function of *t1_lib.c* evolved across three versions of OpenSSL. This function is reported as a vulnerability (CVE-2014-3567) that affects 0.9.8zb and 1.0.1i, and then is fixed in the later version 1.0.1l. As can be seen, compared to Fig.2(a), the fragment in Fig.2(b) adds 4 lines of code (lines 2–5) and changes 1 line (line 11), a total of 5 ...

Implementation details of VDSimilar

In this section, we will describe several technical details, including data preparation, detection model setup, and vulnerability detection. ...

Data set

We evaluate the proposed idea on a set of OpenSSL and Linux vulnerabilities. Since several CVEs affect only a few versions, leading to a few pairs whose account is insufficient for training an accurate model. Therefore, we only consider the CVEs that have more than 5 vulnerable functions and 5 patched functions, so that the number of generated similar pairs and different pairs is sufficient for training. This leaves us with 56 Linux CVEs and 10 OpenSSL CVEs, which fall into 14 CWEs, as shown in ...

Discussion and limitation

Compared to existing methods that builds deep learning models on a large data set, we first prepare a relatively small data set. The data set contain a set of vulnerability functions and corresponding patched functions, which helps characterize vulnerability snippets. Then, we perform detection with Siamese network combined with BiLSTM and attention to learn similarity in the view of vulnerability snippets. The main advantage of our work over state-of-the-art lies in that it performs ...

Conclusions

In this paper, we present a code similarity based vulnerability detection approach. Compared to current work that training a classifier on vulnerabilities, VDSimilar is trained on pairs of vulnerabilities and patches with the aim to compute similarity between two pieces of codes. Therefore, it can better describe similar code for vulnerabilities. In addition, with a metric learning model, our approach can detect newly emerged vulnerabilities without retraining. The experimental results show ...

CRediT authorship contribution statement

Hao Sun: Conceptualization, Methodology, Resources, Writing – original draft. **Lei Cui:** Investigation, Resources, Writing – review & editing, Supervision. **Lun Li:** Investigation. **Zhenquan Ding:** Supervision. **Zhiyu Hao:** Writing – review & editing, Supervision. **Jiancong Cui:** Resources. **Peng Liu:** Investigation. ...

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. ...

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (Grant nos. 61972392, 62072453) and Youth Innovation Promotion Association of the Chinese Academy of Sciences (Grant no. 2020164). ...

Hao Sun received the B.S. degree in software engineering from Harbin University of Science and Technology in 2019. She is currently pursuing the Ph.D. degree with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China. Her research interests include network security, malicious code detection and deep learning. ...

...

...

[Recommended articles](#)

References (35)

A. Graves *et al.*

[Framewise phoneme classification with bidirectional LSTM and other neural network architectures](#)

Neural Netw. (2005)

H. Liang *et al.*

[FIT: inspect vulnerabilities in cross-architecture firmware by deep learning and bipartite matching](#)

Comput. Secur. (2020)

S. Chakraborty *et al.*

Deep learning based vulnerability detection: are we there yet?

IEEE Trans. Softw. Eng. (2020)

S. Chopra *et al.*

Learning a similarity metric discriminatively, with application to face verification

2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05) (2005)

H. Feng *et al.*

Efficient vulnerability detection based on abstract syntax tree and deep learning

IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS) (2020)

A. Graves *et al.*

Hybrid speech recognition with deep bidirectional LSTM

2013 IEEE Workshop on Automatic Speech Recognition and Understanding(2013)

G. Grieco *et al.*

Toward large-scale vulnerability discovery using machine learning

Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy(2016)

Y. He *et al.*

Vul-mirror: a few-shot learning method for discovering vulnerable code clone

EAI Endorsed Trans. Secur. Saf. (2020)

S. Hochreiter *et al.*

Long short-term memory

Neural Comput. (1997)

J.W. Hunt *et al.*

An Algorithm for Differential File Comparison

(1976)



View more references

Cited by (76)

[Survey of source code vulnerability analysis based on deep learning](#)

2025, Computers and Security

Citation Excerpt :

...Experimental findings demonstrate that VulDetector surpasses the efficiency of existing methods. Sun et al. (2021) introduced VDSimilar, a novel code vulnerability detection method based on metric learning, which aims to directly capture similarities from a vulnerability perspective. VDSimilar employs a classification model trained to compute similarity scores through metric learning, effectively differentiating between similar pairs (i.e., two vulnerable functions, labeled as '1') and dissimilar pairs (i.e., a vulnerable function alongside its patched counterpart, labeled as '0')....

[Show abstract](#) ✓

[Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding](#)

2024, Expert Systems with Applications

[Show abstract](#) ✓

[VDoTR: Vulnerability detection based on tensor representation of comprehensive code graphs](#)

2023, Computers and Security

Citation Excerpt :

...One representative study is to extract more semantically-focused program snippets from source code and transform them into vectors using embedding methods, and finally use a bidirectional LSTM neural network to make predictions (Li et al., 2018; Zou et al., 2019). VDSimilar (Sun et al., 2021) focuses on the similarity between vulnerable functions and patched functions and then uses the Siamese network and Attention network for detection. Based on internal vulnerability features and external vulnerability features, Zheng et al. (2022) use a time series neural network algorithm for multiple-type overflow vulnerability prediction....

[Show abstract](#) ✓

[HyVulDect: A hybrid semantic vulnerability mining system based on graph neural network](#)

2022, Computers and Security

[Show abstract](#) ✓

[A Systematic Literature Review on Automated Software Vulnerability Detection Using Machine Learning](#) ↗

2024, ACM Computing Surveys

[Interpreters for GNN-Based Vulnerability Detection: Are We There Yet?](#) ↗

2023, Issta 2023 Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis



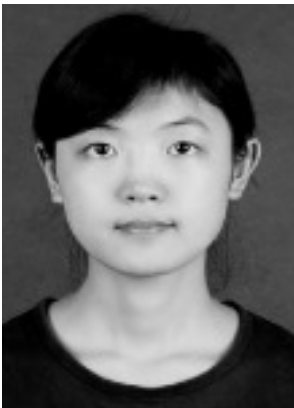
[View all citing articles on Scopus](#) ↗



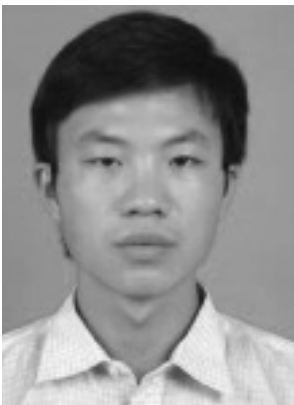
Hao Sun received the B.S. degree in software engineering from Harbin University of Science and Technology in 2019. She is currently pursuing the Ph.D. degree with the School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China. Her research interests include network security, malicious code detection and deep learning.



Lei Cui is an associate professor of Institute of Information Engineering, Chinese Academy of Sciences. He received his Doctor's degree in Computer Software and Theory from Beihang University in 2015. His research interests include operating system, distributed systems and system virtualization. He has published over 20 papers in journals and conferences including VEE, LISA, DSN, The Computer Journal, TPDS.



Lun Li is currently a senior engineer of Institute of Information Engineering, Chinese Academy of Sciences. She received her doctor's degree in Information Security from University of Chinese Academy of Sciences in 2019. Her research interests include network security, system virtualization and network emulation. She has published over 6 papers in journals and conferences including ICA3PP, ICPADS, HPCC.



Zhenquan Ding is currently a associate professor of Institute of Information Engineering, Chinese Academy of Sciences. He received his Master's degree in Computer Science and Technology from Harbin Institute of Technology in 2012. His research interests include cyberspace security, system virtualization and network emulation. He has published many papers in conferences including ICA3PP, HPCC and PDCAT. He has applied for over 20 invention patents and granted 9 patents.



Zhiyu Hao is currently a professor of Institute of Information Engineering, Chinese Academy of Sciences. He received his Doctor's degree in Computer System Architecture from Harbin Institute of Technology in 2007. His research interests include network security, system virtualization and network emulation. He has published over 30 papers in journals and conferences including ICPP, IEEE S&P, ICA3PP and CLUSTER.



Jiancong Cui is currently a guest student of Institute of Information Engineering, Chinese Academy of Sciences. He is an undergraduate of Shandong Normal University (2017-). His research interest is in the area of machine learning and network security.



Peng Liu received his Ph.D. degree from the Beihang University, China in 2017. He joined the Guangxi Normal University as an assistant professor in 2007. Since 2015, he has been an associate professor. His current research interests include network security, data privacy, and graph mining.

[View full text](#)

© 2021 Elsevier Ltd. All rights reserved.



All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

